



INSTITUTO POLITÉCNICO NACIONAL

UNIDAD PROFESIONAL INTERDISCIPLINARIA EN
INGENIERÍA Y TECNOLOGÍAS AVANZADAS

PROGRAMACIÓN AVANZADA

Práctica 1

Alumno:

Martínez Martínez Nicolás

Boleta:

2025640187

Profesor:

Jose Luis Cruz Mora

2MV4

23 de septiembre de 2025

Índice

1. Objetivo	4
2. Introducción	4
3. Desarrollo	5
3.1. ¿Qué es POO?	5
3.2. Elementos de la POO en el programa	5
3.3. Abstracción y Clases Abstractas en Python	5
4. Aplicación en el programa	6
4.1. Abstracción	6
4.2. Atributos	6
4.3. Métodos	6
4.4. Herencia	7
4.5. Polimorfismo	7
4.6. Importación de clases	7
4.7. Menú	7
4.7.1. Función del Programa	8
4.8. Ingreso de datos y almacenamiento de objetos	8
4.9. Regreso al menú principal	9
4.10. Ver mascotas	10
4.11. Finalización del programa	10
5. Resultados	12
Bibliografía	15

Índice de figuras

1.	Menú Principal del programa	12
2.	Menú para seleccionar la mascota	12
3.	Proceso para agregar una mascota doméstica	13
4.	Proceso para agregar una mascota exótica	13
5.	Opción para regresar al menú principal	13
6.	Visualización de las mascotas registradas	14
7.	Salida del programa	14

1. Objetivo

Desarrollar programas utilizando el paradigma orientado a objetos.

2. Introducción

La programación orientada a objetos (POO) es una forma de programar que organiza el código en objetos, los cuales tienen características y acciones. Este método ayuda a que los programas sean más fáciles de entender, modificar y ampliar. En este proyecto se usan los conceptos básicos de la POO para representar distintos tipos de mascotas, creando clases y métodos que muestran sus propiedades y comportamientos. Con esto se busca practicar y reforzar el uso de este paradigma en el desarrollo de programas.

3. Desarrollo

3.1. ¿Qué es POO?

La Programación Orientada a Objetos (POO), conocida en inglés como Object-Oriented Programming (OOP), es un paradigma de programación que surgió en la década de 1970, aunque alcanzó gran popularidad varios años después.

Este enfoque permite organizar el código de una forma similar a como entendemos los objetos en la vida real, utilizando el concepto de clases. Una clase agrupa características (atributos) y funcionalidades (métodos) que definen un determinado objeto.

Por ejemplo, un perro puede representarse como una **clase**. Sus atributos podrían ser la edad, el nombre o la raza, mientras que sus métodos serían acciones como ladrar o correr. A partir de esa clase, es posible crear objetos específicos, como un perro llamado Toby o uno llamado Laika. La clase representa el concepto general de perro, mientras que los objetos son sus instancias particulares [1].

3.2. Elementos de la POO en el programa

La **herencia** es uno de los pilares fundamentales de la Programación Orientada a Objetos (POO). Permite que una clase hija derive atributos y métodos de una clase padre. Gracias a esto se promueve la **reutilización de código** y se simplifica el mantenimiento, facilitando la construcción de programas escalables y eficientes [2].

El **polimorfismo** proviene del griego y significa “muchas formas”. En programación, se refiere a la capacidad de un mismo método o función de comportarse de distintas maneras según el objeto con el que trabaje. En Python puede implementarse de dos formas principales:

- **Polimorfismo con herencia:** cuando las clases hijas sobrescriben métodos de la clase padre.
- **Polimorfismo en funciones:** cuando una función puede aceptar distintos tipos de objetos y ejecutar operaciones específicas según el tipo recibido[3].

Finalmente, el **encapsulamiento** consiste en ocultar los detalles internos de un objeto, limitando el acceso directo a sus atributos y métodos. El objetivo es proteger el estado interno del objeto y permitir que solo sea modificado a través de métodos controlados. Esto se logra mediante modificadores de acceso, que determinan qué partes de un objeto pueden ser visibles o modificables desde fuera de su clase [4].

3.3. Abstracción y Clases Abstractas en Python

La **abstracción** consiste en representar una entidad compleja mediante sus características más relevantes, ocultando los detalles innecesarios. Es como crear una versión simplificada de algo, manteniendo solo lo que necesitamos para interactuar con ello. Esto permite trabajar con objetos de forma más intuitiva y eficiente.[5]

En POO, los **objetos** son la unidad básica de abstracción. Cada objeto tiene **atributos** (propiedades) y **métodos** (acciones que puede realizar). Por ejemplo, en un juego, un personaje

puede abstraerse como un objeto con atributos como nombre, nivel, puntos de vida y métodos como moverse o atacar.

Las **clases abstractas** son un mecanismo para definir abstracciones más formales. Una clase abstracta no puede instanciarse directamente y define métodos que deben implementarse obligatoriamente en cualquier clase que la herede. En Python, esto se logra usando el módulo `abc` y decorando los métodos abstractos con `@abstractmethod`. Los métodos abstractos no contienen implementación, solo indican qué métodos deben existir en las clases hijas. [6]

4. Aplicación en el programa

4.1. Abstracción

Importamos la biblioteca que nos permite tener clases y métodos abstractos usando `from abc import ABC, abstractmethod`.

Luego, definimos la clase abstracta **Mascota** de la siguiente manera: `class Mascota(ABC)`

4.2. Atributos

En Python, podemos hacer que los atributos de un objeto sean privados usando un guion bajo antes de su nombre. Por ejemplo, en la clase **Mascota**:

```
self._nombre = nombre self._edad = edad self._dueno = dueno self._tipo = tipo
```

Esto indica que no se deben usar directamente desde fuera de la clase. Para acceder o cambiar estos valores, usamos métodos como `getters` y `setters`, protegiendo así la información del objeto.

4.3. Métodos

En Python, un método normal, como `getedad`, tiene una implementación completa y devuelve un valor concreto:

Listing 1: Método normal

```
1 def get_edad(self) -> int:
2     return self._edad
```

Por otro lado, un método abstracto, como `Habla`, no tiene implementación. Solo indica que cualquier clase que herede de esta debe definir su propio comportamiento para ese método:

Listing 2: Método abstracto

```
1 @abstractmethod
2 def Habla(self) -> str:
3     pass
```

Esto permite que diferentes clases hijas puedan dar su propia versión del método.

4.4. Herencia

En Python, para indicar que una clase hereda de otra usamos paréntesis después del nombre de la clase:

```
1 class Domestica(Mascota):
```

Esto significa que `Domestica` es la clase hija y `Mascota` es la clase madre. La clase hija hereda los atributos y métodos de la clase madre.

Dentro del constructor, usamos `super()` para llamar al constructor de la clase madre y así inicializar los atributos que se heredan:

```
1 super().__init__(nombre, edad, dueño, tipo)
```

4.5. Polimorfismo

El método `Habla()` está definido como abstracto en la clase `Mascota`. Cada clase hija implementa este método con su propio comportamiento. Por ejemplo, las clases `Gato` y `Perro` lo sobrescriben para que suenen diferente:

```
1 class Gato(Domestica):
2     def Habla(self) -> str:
3         return "Miau soy un Gato"
4
5 class Perro(Domestica):
6     def Habla(self) -> str:
7         return "Guau soy un Perro"
```

Así, aunque todos los objetos tienen un método llamado `Habla()`, el resultado cambia según la clase del objeto que lo llama. Esto es polimorfismo: un mismo método, múltiples comportamientos [3].

4.6. Importación de clases

Para poder utilizar las clases definidas en otro archivo, como `animales.py`, se importan en el programa principal usando la instrucción `from ... import ...`:

```
1 from animales import Gato, Perro, Vibora, ...
```

Con esto, podemos crear instancias de estas clases y trabajar con sus atributos y métodos directamente en la main.

4.7. Menú

Para organizar mejor el programa, se definieron funciones que muestran los menús principales y el submenú de agregar mascotas.

```
1 def menu_principal():
2     print("\n--- MENU PRINCIPAL ---")
3     print("1. Agregar mascota")
4     print("2. Ver mascotas")
5     print("3. Salir")
6
7 def menu_agregar():
8     print("\n--- AGREGAR MASCOTA ---")
9     print("1. Gato")
10    print("2. Perro")
11    ...
12    print("8. Volver al menu principal")
```

Estas funciones se utilizan dentro de un ciclo `while True` para que los menús se repitan continuamente, permitiendo al usuario interactuar con el programa varias veces sin reiniciarlo.

4.7.1. Función del Programa

La función `main()` es el núcleo del programa. Aquí inicializamos la lista `mascotas` que almacenará todos los objetos creados por el usuario.

Dentro de `main()`, se llama primero a la función `menu_principal()` para mostrar las opciones principales. Según la elección del usuario, se puede ingresar al submenú de agregar mascotas mediante la función `menu_agregar()` y así capturar los datos de cada mascota que se desee agregar; y además, este diseño permite que los menús se repitan continuamente.

```
1 def main():
2     mascotas = []
3
4     while True:
5         menu_principal()
6         opcion = input("Elige una opcion: ")
7
8         if opcion == "1":
9             while True:
10                 menu_agregar()
11                 op2 = input("Elige que mascota agregar: ")
```

4.8. Ingreso de datos y almacenamiento de objetos

En esta sección del programa se capturan los datos del usuario para crear un nuevo objeto, en este caso de tipo `Gato` y almacenarlo en la lista `mascotas`:

```
1 if op2 == "1":
2     nombre = input("Nombre del gato: ")
3     edad = int(input("Edad: "))
4     duenio = input("Dueno: ")
```



```

5     tipo = "Domestico"
6     ft = int(input("Que tan tierno es (1-10): \n"))

```

1. Se solicita al usuario el nombre, edad, dueño y el factor que puede ser de ternura o peligro según sea el caso.
2. La función `input()` captura el valor ingresado y `int()` convierte los valores numéricos a enteros.
3. La variable `tipo` se asigna directamente, puede ser como "Domestico" o .Exótico dependiendo el caso.

```

1     p = Gato(nombre, edad, duenio, tipo, ft)
2     p.setFt(ft)

```

1. Se crea un objeto `Gato` con los datos ingresados.
2. Se utiliza el método `setFt()` para asignar explícitamente el valor al objeto.

```

1     mascotas.append(p)
2     print(f"Gato agregado. Total mascotas: {len(mascotas)}")

```

1. El objeto creado se agrega a la lista `mascotas` mediante `append()`, lo que permite mantener un registro de todas las mascotas ingresados.
2. `len(mascotas)` devuelve el número total de objetos actualmente almacenados en la lista, lo que se muestra al usuario para confirmar el registro.

4.9. Regreso al menú principal

En el submenú de agregar mascotas se incluye una opción para volver al menú principal:

```

1     elif op2 == "8":
2         break
3     else:
4         print("Opcion invalida. Intenta de nuevo.")

```

1. Cuando el usuario selecciona la opción 8, se ejecuta la instrucción `break`, la cual termina el bucle `while` del submenú.
2. Al salir del bucle del submenú, el flujo del programa regresa automáticamente al bucle del menú principal, permitiendo que el usuario vea nuevamente las opciones principales.
3. Si el usuario ingresa un valor que no corresponde a ninguna opción, se ejecuta la sección `else` y se muestra un mensaje de error, indicando que la opción es inválida y solicitando un nuevo intento.

4.10. Ver mascotas

Cuando el usuario selecciona la opción para ver las mascotas, el programa primero verifica si la lista `mascotas` está vacía:

```
1 if not mascotas:
2     print("No tienes mascotas aun.")
```

Si la lista no contiene elementos, se muestra un mensaje indicando que aún no hay mascotas registradas.

Si la lista contiene objetos, se ejecuta el bloque `else`:

```
1 else:
2     print("\n--- MIS MASCOTAS ---")
3     for a in mascotas:
4         info = f"Nombre: {a.getnombre()}, Dueño: {a.getdueno()},..."
5         if hasattr(a, "getFt"):
6             info += f", Factor ternura: {a.getFt()}"
7         elif hasattr(a, "getFp"):
8             info += f", Factor peligro: {a.getFp()}"
9         print(info)
```

1. `for a in mascotas`: Este ciclo recorre cada elemento de la lista `mascotas`. Cada objeto se almacena temporalmente en la variable `a` para poder acceder a sus atributos y métodos.
2. Se construye la cadena `info` usando los métodos `getnombre()`, `getdueno()`, `gettipo()` y el método `Habla()` del objeto `a`. Esto permite mostrar información general de cada mascota.
3. La función `hasattr` verifica si un objeto tiene un atributo o método específico[7].
 - Si `a` tiene el método `getFt`, significa que es una mascota doméstica, por lo que se agrega el valor de *factor ternura* a la cadena `info`.
 - Si `a` tiene el método `getFp`, significa que es una mascota exótica, por lo que se agrega el valor de *factor peligro* a `info`.
4. Finalmente, se imprime la cadena completa `info` para mostrar todos los datos de cada mascota, incluyendo los atributos especiales según el tipo de mascota.

4.11. Finalización del programa

Cuando el usuario selecciona la opción de salir (`opcion == "3"`), el programa muestra un mensaje indicando que se cerrará:

```
1 print("Saliendo del programa...")
2 break
```

La instrucción `break` detiene el ciclo `while True`, lo que hace que el programa termine su ejecución y salga del menú principal.

Si el usuario ingresa una opción que no está en el menú, el programa entra en el bloque `else`:

```
1 else:
2     print("Opcion invalida. Intenta de nuevo.")
```

Mostrando un mensaje que indica al usuario que la opción no es válida y solicitando que intente de nuevo.

Finalmente, la condición:

```
1 if __name__ == "__main__":
2     main()
```

Hace que el programa comience a ejecutarse solo si estamos corriendo este archivo directamente. Si el archivo se usa como módulo en otro programa, no se ejecutará automáticamente.

5. Resultados

A continuación se presentan las capturas de pantalla que muestran el funcionamiento del programa. Cada imagen irá acompañada de una breve explicación para describir lo que sucede en esa parte de la ejecución.

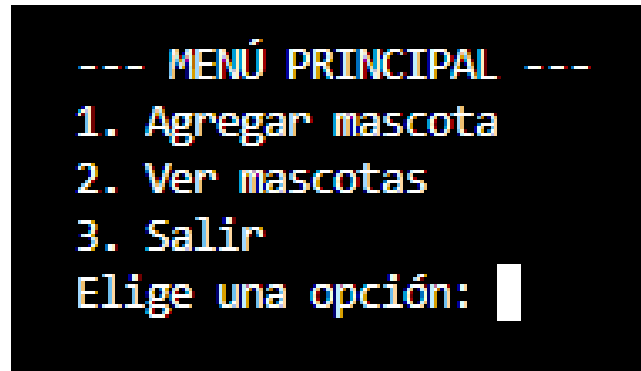


Figura 1: Menú Principal del programa

En la Figura 1 se observa el menú principal del sistema, donde el usuario puede elegir entre agregar una mascota, ver las registradas o salir del programa.

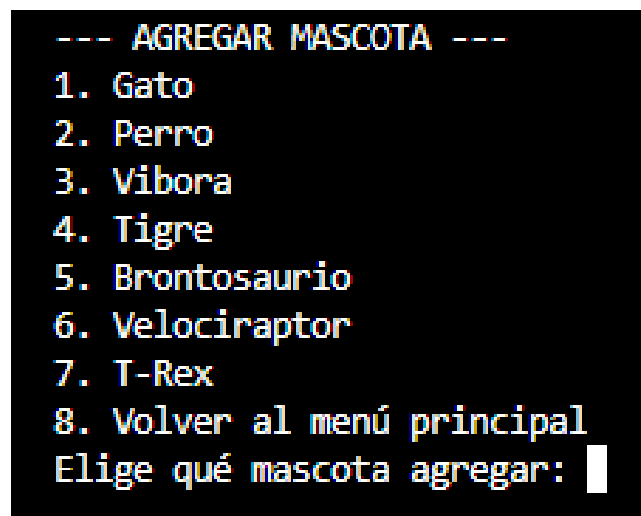


Figura 2: Menú para seleccionar la mascota

La Figura 2 muestra el submenú que aparece al elegir la opción de agregar mascotas.

```
Elige qué mascota agregar: 2
Nombre del perro: Kenny
Edad: 3
Dueño: Nico
Qué tan tierno es (1-10): 10
Perro agregado. Total mascotas: 1
```

Figura 3: Proceso para agregar una mascota doméstica

En la Figura 3 se aprecia cómo el programa solicita los datos para poder registrarla en la lista.

```
Elige qué mascota agregar: 7
Nombre del TRex: Jose
Edad: 150
Dueño: Josue
Qué tan peligroso es (1-10): 10
TRex agregado. Total mascotas: 3
```

Figura 4: Proceso para agregar una mascota exótica

La Figura 4 muestra el proceso similar al anterior, pero en este caso se trata de una mascota exótica, donde se registra el factor de peligro en lugar del de ternura.

```
--- AGREGAR MASCOTA ---
1. Gato
2. Perro
3. Vibora
4. Tigre
5. Brontosaurio
6. Velociraptor
7. T-Rex
8. Volver al menú principal
Elige qué mascota agregar: 8
```

Figura 5: Opción para regresar al menú principal

La Figura 5 corresponde a la opción que permite regresar al menú principal, sin necesidad de reiniciar el programa.

```
--- MIS MASCOTAS ---  
Nombre: Gray, Dueño: Daniel, Tipo: Domestico, Sonido: Miau soy un Gato, Factor ternura: 8  
Nombre: Kenny, Dueño: Nico, Tipo: Domestico, Sonido: Guau soy un Perro, Factor ternura: 10  
Nombre: Pablo, Dueño: Paul, Tipo: Exotico, Sonido: Grrr soy un Tigre, Factor peligro: 10
```

Figura 6: Visualización de las mascotas registradas

En la Figura 6 se presenta la opción de visualizar todas las mascotas guardadas. El programa recorre la lista y muestra los datos de cada una junto con su sonido característico.

```
--- AGREGAR MASCOTA ---  
1. Gato  
2. Perro  
3. Vibora  
4. Tigre  
5. Brontosaurio  
6. Velociraptor  
7. T-Rex  
8. Volver al menú principal  
Elige qué mascota agregar: 8
```

Figura 7: Salida del programa

Finalmente, en la Figura 7 se muestra la opción de salir, la cual termina la ejecución del programa.

Bibliografía

- [1] E. L. de Python, *Programación Orientada a Objetos en Python*, <https://ellibrodepython.com/programacion-orientada-a-objetos-python>, [En línea; accedido: 17-sept-2025], 2025.
- [2] DataCamp, *Python Inheritance Tutorial*, <https://www.datacamp.com/es/tutorial/python-inheritance>, [En línea; accedido: 17-sept-2025], 2025.
- [3] Oregoom, *Polimorfismo en Python*, <https://oregoom.com/python/polimorfismo/>, [En línea; accedido: 17-sept-2025], 2025.
- [4] Oregoom, *Encapsulamiento en Python*, <https://oregoom.com/python/encapsulamiento/>, [En línea; accedido: 17-sept-2025], 2025.
- [5] Programacion.top, *Abstracción en programación orientada a objetos*, <https://programacion.top/orientada-objetos/abstraccion/>, [En línea; accedido: 17-sept-2025], 2025.
- [6] Oregoom, *Abstracción en Python*, <https://oregoom.com/python/abstraccion/>, [En línea; accedido: 17-sept-2025], 2025.
- [7] GeeksforGeeks, *Python hasattr() Method*, <https://www.geeksforgeeks.org/python/python-hasattr-method/>, [En línea; accedido: 17-sept-2025], 2025.